Received: 9 January 2017 | Revised: 10 April 2017 | Accepted: 11 May 2017

DOI: 10.1002/cpe.4208

WILEY

SPECIAL ISSUE PAPER

Scheduling real-time bag-of-tasks applications with approximate computations in SaaS clouds

Georgios L. Stavrinides | **Helen D. Karatza**

Department of Informatics, Aristotle University of Thessaloniki, Thessaloniki, Greece

Correspondence
Georgios L. Stavrinides, Department of Informatics, Aristotle University of Thessaloniki, Thessaloniki 54124, Greece.
Email: gstavrin@csd.auth.gr

Summary

Software as a Service (SaaS) cloud computing has emerged as an attractive platform to tackle various problems of the traditional software distribution model, such as the requirement to acquire and maintain expensive hardware and software infrastructure. SaaS, however, involves many challenges, mainly due to the heterogeneity and multitenancy of the underlying host environment, as well as the nature of the applications executed on such platforms. Applications are usually bags-of-tasks, consisting of independent component tasks that can be executed in any order, featuring different degrees of variability in their computational demands. Furthermore, according to the service level agreement between the cloud provider and the end-users, the execution of such applications must typically complete within a deadline, providing results of acceptable quality. Consequently, one of the most important aspects of SaaS cloud computing is the effective scheduling of multiple parallel applications, avoiding any service level agreement violations. Towards this direction, our contribution in this paper is twofold: (1) We enhance some of the most commonly used scheduling algorithms for bag-of-tasks applications, by utilizing approximate computations, and (2) we investigate the impact of different levels of variability in the computational demands of the applications on the performance of the examined heuristics.

KEYWORDS

approximate computations, bags-of-tasks, computational demand variability, performance evaluation, real-time scheduling, SaaS clouds

1 | INTRODUCTION

Software as a Service (SaaS) cloud computing is becoming a widely adopted paradigm, poised to replace the traditional software distribution model, which requires on-premises software. According to this model, software is provided to the end-users as a service over the Internet, utilizing a particular pricing scheme, such as a pay-as-you-go or a subscription-based approach. SaaS provides better operational efficiency, by eliminating the requirement for upfront investment and ongoing infrastructure maintenance.[1] Furthermore, it provides better scalability, accessibility, and easier administration. The resources of the underlying host environment are usually virtualized, following a *multitenant* approach. That is, applications of different users may share the same virtual machines (VMs).[2] A *service level agreement* (SLA) is a contract between the SaaS cloud provider and the end-users. It defines the type of the provided service, the quality of service (QoS) requirements and the adopted pricing scheme.[3]

Applications in SaaS clouds usually have firm deadlines, within which they must finish execution. Such applications are typically referred to as *real-time*, as the correctness of their output does not only depend on the logical results of the computations but also on the time at which the results are produced. If a real-time application with a firm deadline cannot finish execution within its deadline, then its results will be useless.[4,5] Based on the observation that it is often more desirable for a real-time application to provide an approximate result by its deadline, than a precise result late, Lin et al[6] proposed the technique of *approximate computations*. According to this approach, a real-time application is allowed to return intermediate, approximate results of poorer, but still acceptable quality, when its deadline cannot be met. This technique is especially suitable for *monotone* applications, that is, applications that their results quality is improved as more time is spent to produce them. Statistical estimation and prediction, heuristic search, video and voice transmission, numerical computation, and database query processing are some examples of monotone applications.[7]

 © 2017 John Wiley & Sons, Ltd.

1.1 | Motivation

SaaS, however, involves many challenges, mainly due to the heterogeneity, multitenancy, and contention for resources of the underlying host environment, as well as the nature of the applications executed on such platforms. Applications are usually *bags-of-tasks* (BoT), consisting of independent (ie, embarrassingly parallel) component tasks that can be executed in any order, without intertask communication, featuring different degrees of variability in their computational demands. Examples of BoT applications include data mining, chromosome mapping, pattern matching, and Monte Carlo simulations.[8–10]

Furthermore, according to the SLA between the cloud provider and the end-users, such applications must typically complete within a deadline, providing results of acceptable quality. Consequently, one of the most important aspects of SaaS cloud computing is the effective scheduling of multiple BoT applications, meeting specific QoS requirements and avoiding any SLA violations.

1.2 | Contribution

Towards this direction, our contribution in this paper is twofold: (1) We enhance some of the most commonly used scheduling algorithms for BoT applications, by utilizing approximate computations, and (2) we investigate the impact of different levels of variability in the computational demands of the applications on the performance of the examined heuristics. The performance of the scheduling policies is evaluated via simulation, based on the defined performance metrics.

In our previous work,[11–13] we utilized approximate computations for the scheduling of real-time fine-grained parallel applications (gangs), whereas in Stavrinides and Karatza,[14–17] we employed approximate computations for the scheduling of real-time coarse-grained parallel applications (workflows). To the best of our knowledge, the utilization of approximate computations for the scheduling of real-time embarrassingly parallel applications (bags-of-tasks) in SaaS clouds has never been discussed in the literature before.

The remainder of this paper is organized as follows: Section 2 presents an overview of related work. Section 3 describes the cloud setting under study, the workload model, the utilization of approximate computations, as well as the pricing scheme and the employed scheduling methods. Section 4 defines the performance metrics, describes the experimental setup, and analyzes the simulation results. Finally, Section 5 concludes the paper.

2 | RELATED WORK

Many algorithms have been proposed in the literature for the scheduling of BoT applications, in an attempt to either improve the system performance,[18–21] achieve energy efficiency,[22–25] or minimize the monetary cost required for their execution.[26,27] However, the most widely used BoT scheduling heuristics are: (1) *MinMin*, (2) *MaxMin*, and (3) *Sufferage*. The MinMin and MaxMin policies were first proposed in Ibarra and Kim[28] and implemented in a widely distributed setting in Freund et al.[29] The Sufferage heuristic was first introduced in Maheswaran et al.[30]

The MinMin, MaxMin, and Sufferage scheduling strategies have as a primary goal the minimization of the total execution time of each BoT application. The MinMin heuristic is a two-step iterative process. In the first step, the *minimum completion time* (MCT) of each unassigned task is calculated, over all of the processors in the system. In the second step, the task with the minimum MCT is assigned to the corresponding processor. At each iteration of the algorithm, the MCT of each unassigned task is recalculated, taking into account the current load of the processors, as resulted by the scheduling decision of the previous iteration.[18]

The only difference of the MaxMin algorithm from the MinMin heuristic is in the second step of the scheduling process. Specifically, in this case, the task with the maximum (instead of the minimum) MCT is assigned to the corresponding processor. Consequently, in cases where the application consists of a large number of tasks with small execution times and a few tasks with large execution times, the MaxMin heuristic is likely to give a smaller total execution time than the MinMin algorithm, since it schedules the tasks with larger execution times at earlier iterations. Thus, there is a higher probability that the tasks with smaller execution times that are scheduled at the subsequent iterations will be executed concurrently with the tasks with larger execution times.[20]

According to the Sufferage algorithm, during the first step of the scheduling process, the MCT, the second MCT, and the *sufferage value* of each task are calculated. The sufferage value is equal to the difference between the second MCT and the first MCT of the task. In the second step of the process, the task with the largest sufferage value is assigned to the processor that can provide it with the MCT. This policy is based on the idea that the highest priority for scheduling should be given to the task that would suffer the most (in terms of completion time) if it is not assigned to the processor that can provide it with the MCT.[30] However, none of the proposed algorithms in the literature is focused on improving the scheduling performance of real-time BoT applications in a cloud environment via the utilization of approximate computations.

The impact of variability in the computational demands of applications on the system performance has been examined in other works.[31–35] In Karatza,[31,32] the computational demands of the jobs are considered to be either exponential or hyperexponential (ie, they exhibit moderate or high variability). The workload is either a mixture of fine-grained parallel jobs (gangs) and sequential jobs or consists only of gangs of variable size. No BoT jobs or time constraints are considered. Real-world workload traces are used in Iosup et al,[33] in order to investigate via simulation the variability in the computational demands of BoT. However, using specific traces may introduce bias in the simulation results. Furthermore, no execution deadlines are taken into account. In Zikos and Karatza,[34] exponential and hyperexponential computational demands are considered for single-task jobs, without any deadlines, whereas in Stavrinides and Karatza,[35] non–real-time gang jobs with hyperexponential computational demands and real-time

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

single-task jobs with exponential execution times are considered. However, in contrast to this paper, none of these works consider computational demands that feature a broader spectrum of variability, including low or no variability at all.

3 | PROBLEM FORMULATION

3.1 | Cloud setting

The system under consideration is a SaaS cloud, with an underlying virtualized host environment consisting of a set $V = \{vm_1, vm_2, \ldots, vm_v\}$ of v heterogeneous VMs. Each VM vm_i consists of a single-core vCPU with execution rate μ_i, and it serves its own local queue of tasks (ie, it has its own memory). It is assumed that the VMs are multitenant and fully connected by a virtual network, with contention-free communication. Due to the VM multitenancy, applications of different end-users may share the same VMs. A dedicated VM acts as a central scheduler, responsible for scheduling the component tasks of the application jobs to the other VMs in the cloud.[36] The *heterogeneity degree HD* of the VMs denotes the difference in their processing speed. The execution rate μ_i of each VM vm_i is uniformly distributed in the range $\left[\bar{\mu} \cdot (1 - HD/2), \bar{\mu} \cdot (1 + HD/2)\right]$, where $\bar{\mu}$ is the mean execution rate of the VMs. The target SaaS cloud is shown in Figure 1.

3.2 | Workload model

Since the SaaS cloud is used by multiple end-users to execute their applications, the application jobs arrive dynamically at the central scheduler, according to a Poisson process with rate λ.[37–39] Each job is a BoT, consisting of a set $J = \{t_1, t_2, \ldots, t_n\}$ of n independent parallel tasks, without intertask communication or precedence constraints. Moreover, there are no dependencies between tasks of different jobs. It is assumed that either the component tasks of a job do not require any input data in order to start execution or their required input data are available on all of the VMs in the cloud under study. Furthermore, we assume that the tasks are not preemptible, as preemption of real-time tasks may eventually lead to performance degradation.[40]

Each task t_i in a BoT has a weight w_i, which denotes its *computational volume*, ie, the amount of computational operations that must be executed. This information can be obtained either via profiling or from historical data of previous executions of the particular job.[23] It is assumed that the computational volume of the tasks follows a distribution with a *coefficient of variation CV* and a mean $\overline{w}$. The coefficient of variation *CV* is the ratio of the standard deviation of the computational volume to its mean and determines the variability in the computational demands of the tasks.[35]

The *computational cost Comp* of the task t_i on a VM vm_j is defined as

$$Comp(t_i, vm_j) = w_i / \mu_j, \tag{1}$$

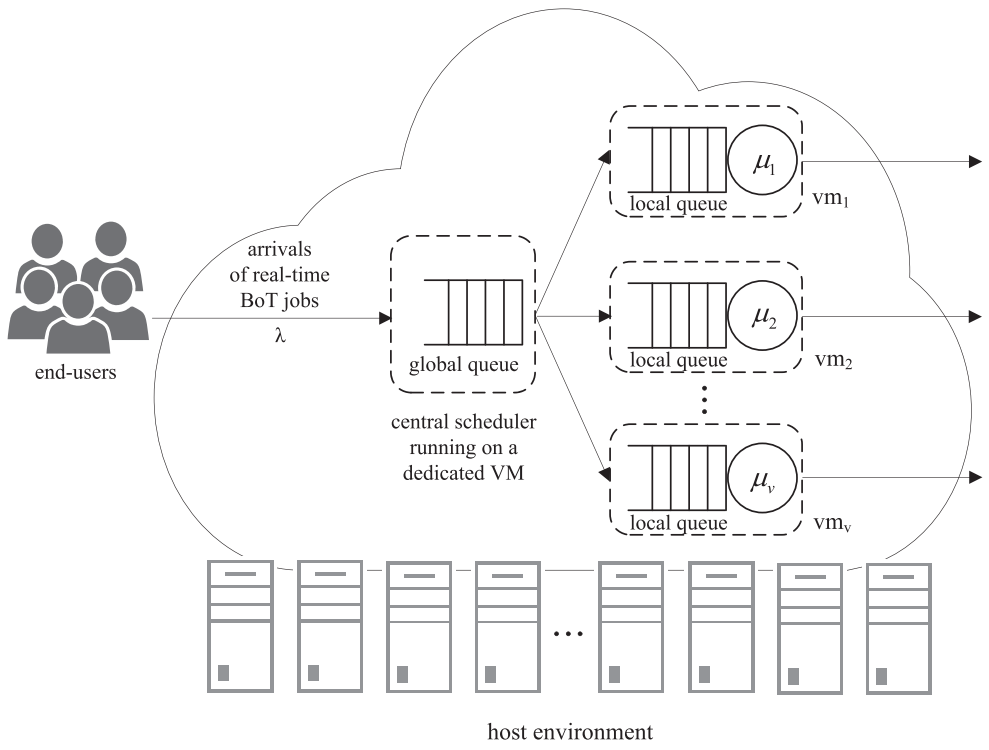


FIGURE 1 The Software as a Service (SaaS) cloud under study. BoT, bags-of-tasks; VM, virtual machine

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

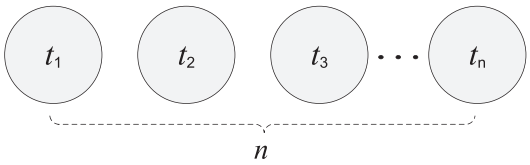


FIGURE 2 A bag-of-tasks job, consisting of n independent parallel tasks

where μ_j is the execution rate of vm_j. The *average computational cost* $\overline{Comp}$ of the task t_i over all of the VMs in the cloud is given by

$$\overline{Comp(t_i)} = \frac{\sum_{vm_j \in V} Comp(t_i, vm_j)}{v}, \tag{2}$$

where v is the number of the VMs in the underlying host environment.

The *length* L of a job is equal to the largest average computational cost of its component tasks. That is,

$$L = \max_{t_i \in J} \left\{ \overline{Comp(t_i)} \right\}. \tag{3}$$

The *makespan* M of a job is its total execution time, and it is defined as

$$M = FT - ST, \tag{4}$$

where FT is the finish time of the job, whereas ST is its start time.

Each job has a *firm end-to-end deadline* D, which is given by

$$D = AT + RD, \tag{5}$$

where AT is the arrival time of the job. RD is the *relative deadline* of the job and is uniformly distributed in the range $[2L, 4L]$. That is, the component tasks of the job do not have individual deadlines, but there is an overall deadline within which all of the tasks must finish execution. When a job fails to complete its execution within its deadline, it is considered lost (ie, the deadline of each job is firm). This is a reasonable assumption, as firm deadlines are frequently considered in the relevant literature,[41,42] as well as in real-world SaaS cloud applications, such as those concerning financial services. Figure 2 depicts an example of a BoT job, consisting of n independent parallel tasks.

3.3 | Utilization of approximate computations

It is assumed that every job consists of component tasks that are *monotone*. The computational volume w_i of each task t_i is assumed to consist of a *mandatory part* mp_i, followed by an *optional part* op_i, as follows:

$$w_i = mp_i + op_i, \tag{6}$$

where $0 < mp_i < w_i$.

A task is completed when at least its mandatory part has been completed. Its optional part refines the result produced by its mandatory part. The optional part can be completed either fully, partially, or it may be entirely skipped. The results of a partially completed task (ie, a task that has not completed its whole optional part) are *approximate*. Consequently, the task has *output error*. Since each task is monotone, its output error is a monotone nonincreasing function of the processed fraction of its optional part. Without loss of generality, it is assumed that the output error of a task t_i is defined by the following linear function[7]:

$$OE_i = \frac{op_i - \phi i}{op_i}, \tag{7}$$

where ϕ_i is the processed fraction of the optional part op_i of the task. As it holds that $0 \leqslant \phi_i \leqslant op_i$, from Equation 7, it follows that $0 \leqslant OE_i \leqslant 1$.

It is assumed that there is a *result precision threshold* RPT, common for all tasks, under which the results of a task are not acceptable. A task provides results of the minimum acceptable quality when only its mandatory part is completed and its entire optional part is omitted. There is a relation between the result precision threshold RPT and the mandatory part mp_i of a task t_i, given by

$$mp_i = RPT \cdot w_i, \tag{8}$$

where w_i is the computational volume of task t_i.

The *result precision* of a task t_i is defined as

$$RP_i = RPT + (1 - RPT)(1 - OE_i), \tag{9}$$

where OE_i is the output error of the task. The result precision of a job is equal to the average result precision of its component tasks.

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

There are cases where the deadline of a job in service is reached, but the job is not considered lost. Specifically, if its uncompleted component tasks have all completed their mandatory part, then the job is considered completed. In this case, however, the overall results of the job are approximate (but still of acceptable quality), since each uncompleted component task gives as a result the one produced at the time of the job's deadline. In any other case, the job is considered lost.

3.4 | Pricing scheme

Software as a Service cloud providers may use various pricing schemes. Among them, pay-as-you-go and subscription-based models are the most commonly used. In this paper, for simplicity reasons and without loss of generality, we consider a pay-as-you-go pricing model, where each end-user is charged a fixed amount per time unit for the execution of each of their jobs. Consequently, the monetary cost required for the execution of a job is proportional to its makespan, ie, its total execution time. It is assumed that according to the SLA between the cloud provider and the end-users, no cost is charged for jobs that fail to meet their deadline, since these jobs are lost and thus the end-users need to resubmit them.

3.5 | Scheduling heuristics

The most widely used policies for the scheduling of BoT are employed: MinMin, MaxMin, and Sufferage, as described in Section 2. However, since these two-step iterative heuristics were designed for non–real-time workloads, we adapted them so that the deadlines of the jobs are taken into account during the scheduling process. Specifically, when a BoT job arrives at the central scheduler (dedicated VM), it enters a global queue and the scheduler is invoked. In the first step of the first iteration of the scheduling process, the *estimated completion time CT* of each component task t_i of the job is calculated on each VM vm_j of the cloud:

$$CT(t_i, vm_j) = T_{idle}(t_i, vm_j) + Comp(t_i, vm_j), \tag{10}$$

where $T_{idle}(t_i, vm_j)$ is the time at which vm_j will be able to execute task t_i. $Comp(t_i, vm_j)$ is the computational cost of task t_i on vm_j.

Algorithm 1 The MinMin, MaxMin, and Sufferage scheduling heuristics.

Input: A BoT job $J = \{t_1, t_2, \dots, t_n\}$ of n independent parallel tasks, a set $V = \{vm_1, vm_2, \dots, vm_v\}$ of v heterogeneous VMs and the applicable Scheduling Policy.

Output: A schedule of J onto V.

1: **repeat**
2: /* 1st Step */
3: **for** each task $t_i \in J$ **do**
4: **for** each VM $vm_j \in V$ **do**
5: $CT(t_i, vm_j) \leftarrow T_{idle}(t_i, vm_j) + Comp(t_i, vm_j)$
6: **end for**
7: $MCT(t_i) \leftarrow \min_{vm_j \in V} \{CT(t_i, vm_j)\}$
8: **if** Scheduling Policy = Sufferage **then**
9: $Suff(t_i) \leftarrow MCT'(t_i) - MCT(t_i)$
10: **end if**
11: **end for**
12: /* 2nd Step */
13: **if** Scheduling Policy = MinMin **then**
14: select $t_i \in J$ so that $MCT(t_i) = \min_{t_m \in J} \{MCT(t_m)\}$
15: assign t_i to vm_j where $MCT(t_i) = CT(t_i, vm_j)$
16: mark t_i as scheduled and remove it from J
17: **else if** Scheduling Policy = MaxMin **then**
18: select $t_i \in J$ so that $MCT(t_i) = \max_{t_m \in J} \{MCT(t_m)\}$
19: assign t_i to vm_j where $MCT(t_i) = CT(t_i, vm_j)$
20: mark t_i as scheduled and remove it from J
21: **else if** Scheduling Policy = Sufferage **then**
22: select $t_i \in J$ so that $Suff(t_i) = \max_{t_m \in J} \{Suff(t_m)\}$
23: assign t_i to vm_j where $MCT(t_i) = CT(t_i, vm_j)$
24: mark t_i as scheduled and remove it from J
25: **end if**
26: **until** all tasks $t_i \in J$ are scheduled

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 The investigated scheduling heuristics

Notation	Task selection criteria	Utilization of approximate computations
MinMin	Task with the minimum *MCT*	No
MinMinMAC	Task with the minimum *MCT*	Moderately
MinMinEAC	Task with the minimum *MCT*	Extensively
MaxMin	Task with the maximum *MCT*	No
MaxMinMAC	Task with the maximum *MCT*	Moderately
MaxMinEAC	Task with the maximum *MCT*	Extensively
Sufferage	Task with the largest *Suff*	No
SufferageMAC	Task with the largest *Suff*	Moderately
SufferageEAC	Task with the largest *Suff*	Extensively

Abbreviation: MCT, minimum estimated completion time.

For the calculation of the $T_{idle}(t_i, vm_j)$ parameter, we first find the potential position of task t_i in vm_j's queue, according to the deadline D of t_i's job, so that the task with the earliest deadline is placed at the head of vm_j's queue. That is, we consider that the local queues of the VMs are arranged according to the *Earliest Deadline First (EDF)* policy.[43] Subsequently, the *minimum estimated completion time MCT* of each task t_i over all of the VMs in the cloud is calculated as follows:

$$MCT(t_i) = \min_{vm_j \in V} \left\{ CT(t_i, vm_j) \right\}. \tag{11}$$

In the case of the Sufferage strategy, at this point, we also find the *second minimum estimated completion time MCT′* of each task t_i, over all of the VMs in the cloud. Furthermore, the *sufferage value Suff* of each task t_i is calculated as

$$Suff(t_i) = MCT'(t_i) - MCT(t_i). \tag{12}$$

In the second step of the first iteration of the scheduling process, we assign to the VM that can provide with the minimum estimated completion time *MCT* the component task that has either:

- the minimum *MCT* among all of the tasks of the job, in the MinMin case,
- the maximum *MCT*, in the MaxMin case, or
- the largest *Suff*, in the Sufferage case.

Subsequently, a second iteration of the scheduling process is performed, taking into account the current load of the VMs as resulted by the scheduling decision of the previous iteration. The iterations of the scheduling process continue, until all of the component tasks of the job are scheduled. The pseudocode for the MinMin, MaxMin, and Sufferage algorithms is shown in Algorithm 1.

For each of the above scheduling heuristics, we provide an alternative version that utilizes approximate computations, either moderately (denoted by the extension MAC in the algorithm's name) or extensively (denoted by the extension EAC in the algorithm's name). Specifically, in the MAC version of each policy, a task is allowed to return approximate results when the deadline of its job is reached and it has completed its mandatory part, as described in Section 3.3. In the EAC version of each strategy, all of the tasks are scheduled to complete only their mandatory part. All of the investigated scheduling heuristics are summarized in Table 1.

4 | EVALUATION

4.1 | Performance metrics

In order to compare the performance of the proposed scheduling heuristics with their respective baseline policies, as well as investigate the impact of computational demand variability on the performance of the algorithms, the following metrics were employed:

- *SLA violation ratio* (*SVR*), which is the ratio of the number of jobs that missed their deadline, over the number of all of the jobs that arrived at the central scheduler.
- *Average result precision* ($\overline{RP}$) of the completed jobs.
- *Average cost per job* ($\overline{Cost}$), which is given by

$$\overline{Cost} = \overline{M} \cdot PPTU, \tag{13}$$

where $\overline{M}$ is the *average makespan* of the completed jobs, whereas *PPTU* is the *price per time unit*, according to the pricing model described in Section 3.4.

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

4.2 | Experimental setup

The performance of the scheduling strategies was evaluated via simulation. Due to the complexity of the framework under study, we implemented our own discrete-event simulation program in C++, tailored to the specific requirements of the particular problem. Discrete-event simulation concerns the modeling of a system as it evolves over time. The system state is represented by variables that change instantaneously at separate points in time (ie, at discrete events).[44] Our custom simulator was based on the next-event time advance approach.[45] Specifically, the simulation clock was first initialized to zero and was then advanced to the time of occurrence of the most imminent event in the future events list. At each event occurrence, the state variables of the system were updated and the occurrence time of future events was determined. The simulation clock was then advanced to the time of occurrence of the next event in the future events list. For each set of input parameters, we conducted 30 runs of the simulation program, utilizing the independent replications method. That is, different seeds of random numbers were used in each run. Each replication was terminated when 10^6 jobs had been completed. For every mean value, a 95% confidence interval was evaluated. The half-widths of all of the confidence intervals were less than 5% of their respective mean values. No specific workload traces were used, but synthetic workload, for the following reasons:

- Due to the particular characteristics of the examined workload, such as the end-to-end deadlines and approximate computations.
- In order to control the level of variability in the computational demands of the component tasks of the jobs, as the investigation of its impact on the scheduling performance is one of the goals of this study.
- In order to obtain unbiased, general results, not applicable only to particular workload traces.

The simulation input parameters are shown in Table 2. The heterogeneity degree of the VMs was chosen to be equal to $HD = 0.5$, as the virtualized computational resources typically feature a moderate degree of heterogeneity.[46] The value of the arrival rate λ of the jobs was chosen so that the system is stable. In the case of approximate computations, the result precision threshold of the tasks was chosen to be equal to $RPT = 0.75$, as a smaller value would give a greater benefit to the heuristics that utilize approximate computations. On the other hand, a larger value would negate the point of using approximate computations.

At the one end of the spectrum, a BoT job may consist of tasks with identical computational demands. At the other end of the spectrum, the component tasks of a BoT job may feature high variability in their computational demands. Consequently, in order to investigate the impact of computational demand variability on the performance of the scheduling strategies, we considered that the coefficient of variation CV of the computational volume of the component tasks of a job is $CV = \{0, 0.5, 1, 1.5\}$. When $CV = 0$, the computational volume is constant for all tasks. That is, all tasks require the same amount of computational operations, equal to the mean $\overline{w}$. In the case where $CV = 1$, the computational volume of the tasks is exponentially distributed with mean $\overline{w}$. In this case, the computational demands exhibit moderate variability. When $0 < CV < 1$, the variability in the computational demands is low, compared to the exponential distribution. In this case, the computational volume of the tasks follows a hypoexponential distribution with mean $\overline{w}$. This means that the values of the computational volume of the tasks are closer to the mean when compared to the exponential distribution. In the case where $CV > 1$, the computational demands of the tasks feature high variability, compared to the exponential distribution. In this case, the computational volume of the tasks follows a hyperexponential distribution with mean $\overline{w}$. This means that there is a large number of tasks with very low computational demands compared to the mean and a comparatively small number of tasks with very high computational demands.

TABLE 2 The input parameters of the simulation model

Parameter	Value
Number of completed jobs	10^6
Number of VMs	$v = 128$
Mean VM execution rate	$\bar{\mu} = 1$
Heterogeneity degree of the VMs	$HD = 0.5$
Number of tasks per job	$n \sim U[1, 64]$
Job arrival rate	$\lambda = 3.25$
Job relative deadline	$RD \sim U[2L, 4L]$
Task result precision threshold	$RPT = 0.75$
Mean task computational volume	$\overline{w} = 1$
Coefficient of variation of task computational volume	$CV = \{0, 0.5, 1, 1.5\}$
Price per time unit	$PPTU = \$0.01$
Scheduling method	MinMin, MinMinMAC, MinMinEAC, MaxMin, MaxMinMAC, MaxMinEAC, Sufferage, SufferageMAC, SufferageEAC

Abbreviation: VM, virtual machine.

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

4.3 | Simulation results

The simulation results, according to the performance metrics defined in Section 4.1, are presented in Figures 3, 4, 5, and 6, with respect to the coefficient of variation *CV* of the computational volume of the component tasks of the jobs. Figure 3 shows the SLA violation ratio *SVR* when each of the investigated scheduling heuristics is employed. It is noted that a logarithmic scale is used for the *SVR* values, as they are highly skewed. Figure 4 illustrates the average result precision $\overline{RP}$ of the completed jobs in each case. The average makespan $\overline{M}$ of the completed jobs and the average cost per job $\overline{Cost}$ for each scheduling policy are shown in Figures 5 and 6, respectively. The analysis of the simulation results is presented for each level of computational demand variability.

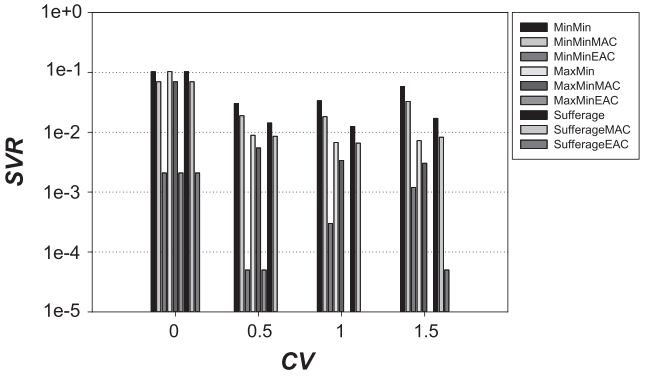


FIGURE 3 The service level agreement violation ratio (*SVR*) vs the coefficient of variation (*CV*) of the computational volume of the component tasks of the jobs

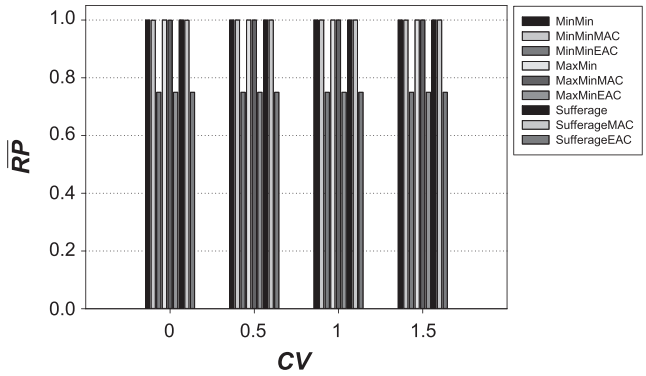


FIGURE 4 The average result precision ($\overline{RP}$) of the completed jobs vs the coefficient of variation (*CV*) of the computational volume of the component tasks of the jobs

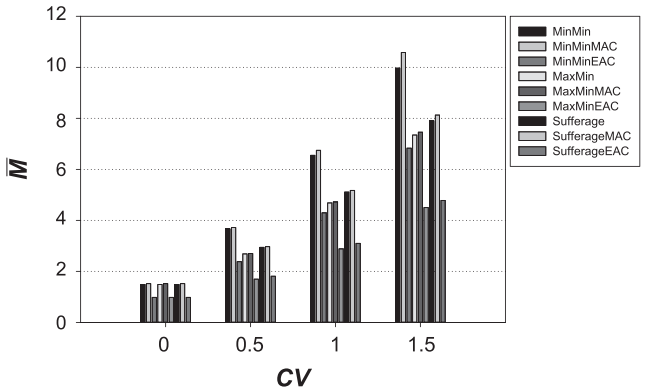


FIGURE 5 The average makespan ($\overline{M}$) of the completed jobs vs the coefficient of variation (*CV*) of the computational volume of the component tasks of the jobs

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

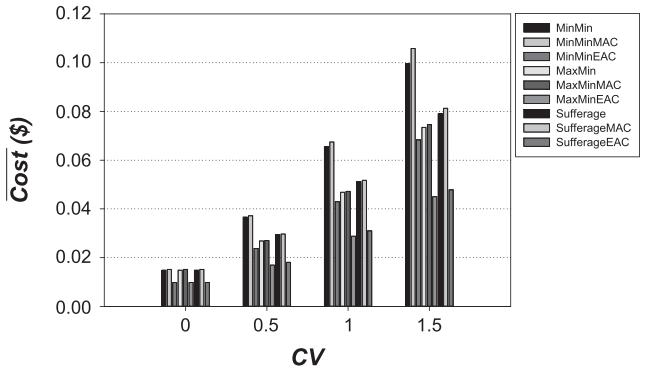


FIGURE 6 The average cost per job ($\overline{Cost}$) vs the coefficient of variation (CV) of the computational volume of the component tasks of the jobs

4.3.1 | Results for $CV = 0$

When the computational volume is constant for all tasks ($CV = 0$), the corresponding version of each scheduling heuristic yields the same performance. In terms of the SLA violation ratio *SVR*, the EAC version of each policy exhibits the best performance, with an improvement of 97.98% over the corresponding baseline version. However, when this version of the algorithms is employed, there is a decrease of 25% in the average result precision $\overline{RP}$, compared to the baseline versions. The MAC version of each policy gives an improvement of 32.29% in the SLA violation ratio, over the corresponding version of each strategy that does not utilize approximate computations. The decrease in the average result precision in this case is very small (0.06%), compared to the baseline versions. The versions of the algorithms that do not utilize approximate computations, even though they always give precise results, they exhibit the highest SLA violation ratio.

In terms of the average makespan $\overline{M}$ and the average cost per job $\overline{Cost}$ metrics, the EAC versions of the algorithms outperform the other policies, since in this case the component tasks of the jobs are scheduled to complete only their mandatory part. Compared to the baseline versions of the algorithms, the improvement in this case is 33.88%. The MAC versions of the strategies give a slightly larger average makespan (2.32%), and thus a slightly larger monetary cost, compared to the baseline heuristics. This is due to the fact that they take into account jobs that reach their deadline and their tasks have completed their mandatory part. In the case of the baseline algorithms, these jobs would be considered lost and thus not taken into account in the calculation of the average makespan.

4.3.2 | Results for $CV = 0.5$

When the computational demands of the tasks feature low variability ($CV = 0.5$), the EAC versions of the scheduling heuristics outperform all of the other policies, in terms of the SLA violation ratio *SVR*, average makespan $\overline{M}$ and average cost per job $\overline{Cost}$ metrics. Specifically, in terms of the *SVR* parameter, MinMinMAC and MinMinEAC give an improvement of 37.67% and 99.84%, respectively, compared to the baseline MinMin policy. Compared to the MaxMin heuristic, the alternative versions that utilize approximate computations, MaxMinMAC and MaxMinEAC, exhibit an improvement of 38.68% and 99.44%, respectively. The SufferageMAC and SufferageEAC policies yield an improvement of 39.99% and 100%, respectively, compared to the Sufferage heuristic. The SufferageEAC policy does not give any SLA violations, whereas the MinMin policy yields the highest SLA violation ratio.

The EAC versions of the policies yield the lowest average result precision $\overline{RP}$, whereas the baseline versions the highest. Specifically, the EAC versions exhibit a decrease of 25%, compared to the baseline versions. The MAC versions give slightly approximate results. Specifically, the average result precision is decreased by 0.004%, 0.0015%, and 0.0021% in the case of the MinMinMAC, MaxMinMAC, and SufferageMAC policies, respectively, when compared to the baseline MinMin, MaxMin, and Sufferage strategies.

As in the case where $CV = 0$, the MAC version of the policies yields a slightly larger average makespan and thus a slightly larger average cost, compared to the baseline versions. On the other hand, the EAC version of the algorithms yields better results. Specifically, in the case of MinMinMAC, MaxMinMAC, and SufferageMAC, the increase is 1.51%, 0.43%, and 0.92%, when compared to MinMin, MaxMin, and Sufferage, respectively. In the case of MinMinEAC, MaxMinEAC, and SufferageEAC, compared to their corresponding baseline strategies, the decrease is 35.04%, 36.84%, and 38.45%, respectively. Overall, MaxMinEAC yields the smallest average makespan and thus the lower average cost, whereas MinMinMAC the largest.

4.3.3 | Results for $CV = 1$

In the case where the computational volume of the tasks exhibits moderate variability ($CV = 1$), the alternative versions of the investigated policies that utilize approximate computations outperform their respective baseline versions, in terms of the SLA violation ratio *SVR* metric. Specifically, MinMinMAC and MinMinEAC give an improvement of 45.75% and 99.11%, respectively, compared to MinMin. MaxMinMAC and MaxMinEAC exhibit an improvement of 49.80% and 100%, respectively, compared to the MaxMin heuristic. Compared to Sufferage, the SufferageMAC and SufferageEAC policies yield an improvement of 47.13% and 100%, respectively. It can be observed that the MaxMinEAC and SufferageEAC strategies do not give any SLA violations. On the other hand, the MinMin policy yields the highest SLA violation ratio.

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

As in the other cases of *CV* values, the EAC versions of the policies give the lowest average result precision $\overline{RP}$, whereas the baseline versions the highest. Specifically, the EAC versions exhibit a decrease of 25%, compared to the baseline versions. The MAC versions give almost precise results, with a decrease in the average result precision of 0.0052%, 0.0009%, and 0.0021% in the case of the MinMinMAC, MaxMinMAC, and SufferageMAC policies, when compared to their corresponding baseline versions, MinMin, MaxMin, and Sufferage, respectively.

In terms of the average makespan $\overline{M}$ and average cost per job $\overline{Cost}$ metrics, MinMinMAC gives an increase of 3.03% and MinMinEAC a decrease of 34.46%, compared to the MinMin policy. Compared to the MaxMin policy, MaxMinMAC gives an increase of 0.79% in the average makespan, whereas MaxMinEAC yields a decrease of 38.49%. There is an increase of 1.27% in the case of SufferageMAC and a decrease of 39.36% in the case of SufferageEAC, when compared to the baseline Sufferage policy. Overall, the MaxMinEAC heuristic gives the smallest average makespan and thus the lowest monetary cost. On the other hand, MinMinMAC exhibits the worst performance according to these metrics.

4.3.4 | Results for $CV = 1.5$

When the variability of the computational demands of the component tasks is high ($CV = 1.5$), in terms of the SLA violation ratio *SVR* parameter, MinMinMAC and MinMinEAC yield an improvement of 43.79% and 97.94%, respectively, compared to the MinMin policy. MaxMinMAC and MaxMinEAC give an improvement of 58.03% and 100%, respectively, when compared to the MaxMin strategy. Compared to Sufferage, the SufferageMAC and SufferageEAC policies give an improvement of 51.59% and 99.71%, respectively. There are no SLA violations when the MaxMinEAC heuristic is employed. On the other hand, the SLA violation ratio is the highest in the case of the MinMin policy.

As in the other cases of *CV* values, the EAC versions of the algorithms yield the lowest average result precision $\overline{RP}$, with a decrease of 25%, when compared to the baseline versions, which always give precise results. The MAC versions give slightly approximate results, with a decrease in the average result precision of 0.0091%, 0.0016%, and 0.003% in the case of MinMinMAC, MaxMinMAC, and SufferageMAC, when compared to their corresponding baseline versions, MinMin, MaxMin, and Sufferage, respectively.

In terms of the average makespan $\overline{M}$ and average cost per job $\overline{Cost}$ metrics, MinMinMAC gives an increase of 6.15%, whereas MinMinEAC gives a decrease of 31.44%, compared to the MinMin heuristic. Compared to MaxMin, MaxMinMAC yields an increase of 1.54%, whereas there is a decrease of 38.77% in the case of the MaxMinEAC policy. There is an increase of 2.93% in the case of SufferageMAC and a decrease of 39.47% in the case of SufferageEAC, when compared to Sufferage. As in the cases where $CV = 0.5$ and $CV = 1$, the MaxMinEAC heuristic gives the smallest average makespan and thus the lowest monetary cost, whereas the MinMinEAC policy the highest.

4.3.5 | General observations

In terms of the SLA violation ratio *SVR* metric, the MaxMin algorithm and especially its alternative versions that utilize approximate computations yield better performance with the increase of the variability in the computational demands of the component tasks of the jobs. Specifically, the version of the algorithm that utilizes approximate computations extensively, MaxMinEAC, for higher levels of variability gives no SLA violations. The promising performance of the MaxMin algorithm and its variations may be explained by the fact that, in contrast to the other policies, they give higher priority to tasks with higher computational demands during the scheduling process. Tasks with smaller execution times are scheduled towards the end of the process. Consequently, there is a higher probability that the tasks with smaller execution times will be executed concurrently with the tasks with larger execution times and thus a higher probability that each job will meet its deadline.

In terms of the average result precision $\overline{RP}$ metric, the increase of the variability in the computational demands of the tasks does not affect significantly the performance of the scheduling heuristics. On the other hand, the average makespan $\overline{M}$ and the average cost per job $\overline{Cost}$ increase as the variability increases, regardless of the employed scheduling strategy. However, the increase in the case of MaxMin and its alternative versions is smaller compared to the other policies. Overall, the proposed alternative versions of the scheduling heuristics generally give promising performance, when compared to their corresponding baseline versions.

5 | CONCLUSIONS

In this paper, we proposed alternative versions that utilize approximate computations (either moderately or extensively) for some of the most commonly used algorithms for the scheduling of BoT applications, namely, MinMin, MaxMin, and Sufferage. The performance of the proposed heuristics was evaluated in an SaaS cloud by simulation, under different levels of variability in the computational demands of the applications. According to the experimental evaluation, the proposed heuristics generally outperformed their corresponding baseline policies.

Furthermore, we investigated the impact of the computational demand variability on the performance of the employed scheduling strategies. It was observed that with the increase of variability in the computational demands of the component tasks of the jobs, the performance of the investigated policies generally degraded, except in the case of the MaxMin algorithm and its variations, where it improved, in terms of the SLA violations ratio metric.

In this paper, it was assumed that either the component tasks of the jobs did not require any input data in order to start execution or their required input data were available on all of the VMs in the cloud under study. Consequently, a future direction of our research is to investigate in this framework the impact of input data locality on the scheduling performance of BoT applications. Moreover, the case where the underlying VMs of the SaaS cloud feature different levels of multitenancy could be explored.[47]

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

REFERENCES

1. Banerjee S, Srikanth H, Cukic B. Log-based reliability analysis of software as a service (saas). In: Proceedings of the 21st IEEE International Symposium On Software Reliability Engineering (ISSRE'10), San Jose, CA; November 2010:239-248.
2. Höfer CN, Karagiannis G. Cloud computing services: taxonomy and comparison. *J Internet Serv Appl*. 2011;2(2):81-94.
3. Zhao Y, Calheiros RN, Gange G, Ramamohanarao K, Buyya R. Sla-based resource scheduling for big data analytics as a service in cloud computing environments. In: Proceedings of the 44th International Conference on Parallel Processing (ICPP'15), Beijing, China; September 2015:510-519.
4. Stavrinides GL, Karatza HD. Scheduling multiple task graphs in heterogeneous distributed real-time systems by exploiting schedule holes with bin packing techniques. *Simul Model Pract Theory*. 2011;19(1):540-552.
5. Stavrinides GL, Karatza HD. The impact of resource heterogeneity on the timeliness of hard real-time complex jobs. In: Proceedings of the 7th International Conference on PErvasive Technologies Related to Assistive Environments (PETRA'14), Workshop on Distributed Sensor Systems for Assistive Environments (Di-Sensa), Rhodes, Greece; May 2014:65:1-65:8.
6. Lin KJ, Natarajan S, Liu JWS. Imprecise results: utilizing partial computations in real-time systems. In: Proceedings of the 8th IEEE Real-Time Systems Symposium (RTSS'87), San Jose, CA; December 1987:210-217.
7. Feng WC, Liu JWS. Algorithms for scheduling real-time tasks with input error and end-to-end deadlines. *IEEE Trans Software Eng*. 1997;23(2):93-106.
8. White SW, Torney DC. Use of a workstation cluster for the physical mapping of chromosomes. *SIAM News*. 1993:14-17.
9. Senger H, Hruschka ER, Silva FAB, Sato LM, Bianchini CP, Jerosch BF. Exploiting idle cycles to execute data mining applications on clusters of pcs. *J Syst Software*. 2007;80(5):778-790.
10. Silva FAB, Senger H. Scalability limits of bag-of-tasks applications running on hierarchical platforms. *J Parallel Distrib Comput*. 2011;71(6):788-801.
11. Stavrinides GL, Karatza HD. Performance evaluation of gang scheduling in distributed real-time systems with possible software faults. In: Proceedings of the 2008 International Symposium on Performance Evaluation of Computer and Telecommunication Systems SPECTS'08, Edinburgh, UK; June 2008:1-7.
12. Stavrinides GL, Karatza HD. Fault-tolerant gang scheduling in distributed real-time systems utilizing imprecise computations. *Simulation: Trans Soc Model Simul Int*. 2009;85(8):525-536.
13. Stavrinides GL, Karatza HD. Scheduling real-time parallel applications in saas clouds in the presence of transient software failures. In: Proceedings of the 2016 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'16), Montreal, Canada; July 2016:1-8.
14. Stavrinides GL, Karatza HD. Scheduling multiple task graphs with end-to-end deadlines in distributed real-time systems utilizing imprecise computations. *J Syst Software*. 2010;83(6):1004-1014.
15. Stavrinides GL, Karatza HD. The impact of input error on the scheduling of task graphs with imprecise computations in heterogeneous distributed real-time systems. In: Proceedings of the 18th International Conference on Analytical and Stochastic Modelling Techniques and Applications (ASMTA'11), Venice, Italy; June 2011:273-287.
16. Stavrinides GL, Karatza HD. Scheduling real-time dags in heterogeneous clusters by combining imprecise computations and bin packing techniques for the exploitation of schedule holes. *Future Gener Comput Syst*. 2012;28(7):977-988.
17. Stavrinides GL, Karatza HD. A cost-effective and qos-aware approach to scheduling real-time workflow applications in paas and saas clouds. In: Proceedings of the 3rd International Conference on Future Internet of Things and Cloud (FICLOUD'15), Rome, Italy; August 2015:231-239.
18. Weng C, Lu X. Heuristic scheduling for bag-of-tasks applications in combination with QoS in the computational grid. *Future Gener Comput Syst*. 2005;21(2):271-280.
19. Lee YC, Zomaya AY. Practical scheduling of bag-of-tasks applications on grids with dynamic resilience. *IEEE Trans Comput*. 2007;56(6):815-825.
20. Tabak EK, Cambazoglu BB, Aykanat C. Improving the performance of independent task assignment heuristics minmin, maxmin and sufferage. *IEEE Trans Parallel Distrib Syst*. 2014;25(5):1244-1256.
21. Papazachos ZC, Karatza HD. Scheduling bags of tasks and gangs in a distributed system. In: 2015 International Conference on Computer, Information and Telecommunication Systems (CITS'15), Gijón, Spain; July 2015:1-5.
22. Kim KH, Buyya R, Kim J. Power aware scheduling of bag-of-tasks applications with deadline constraints on dvs-enabled clusters. In: Proceedings of the 7th IEEE International Symposium on Cluster Computing and the Grid (CCGRID'07), Rio de Janeiro, Brazil; May 2007:541-548.
23. Calheiros RN, Buyya R. Energy-efficient scheduling of urgent bag-of-tasks applications in clouds through dvfs. In: Proceedings of the 6th IEEE International Conference on Cloud Computing Technology and Science (CLOUDCOM'14), Singapore, Singapore; December 2014:342-349.
24. Terzopoulos G, Karatza HD. Power-aware bag-of-tasks scheduling on heterogeneous platforms. *Cluster Comput*. 2016;19(2):615-631.
25. Stavrinides GL, Karatza HD. Simulation-based performance evaluation of an energy-aware heuristic for the scheduling of hpc applications in large-scale distributed systems. In: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE'17), 3rd International Workshop on Energy-aware Simulation (ENERGY-SIM'17), L' Aquila, Italy; 2017.
26. Gutierrez-Garcia JO, Sim KM. A family of heuristics for agent-based elastic cloud bag-of-tasks concurrent scheduling. *Future Gener Comput Syst*. 2013;29(7):1682-1699.
27. Moschakis IA, Karatza HD. A meta-heuristic optimization approach to the scheduling of bag-of-tasks applications on heterogeneous clouds with multi-level arrivals and critical jobs. *Simul Model Pract Theory*. 2015;57:1-25.
28. Ibarra OH, Kim CE. Heuristic algorithms for scheduling independent tasks on nonidentical processors. *J ACM*. 1977;24(2):280-289.
29. Freund RF, Gherrity M, Ambrosius S, et al. Scheduling resources in multi-user, heterogeneous, computing environments with smartnet. In: Proceedings of the 7th Heterogeneous Computing Workshop (HCW'98), Orlando, FL; March 1998:184-199.
30. Maheswaran M, Ali S, Siegel HJ, Hensgen D, Freund RF. Dynamic mapping of a class of independent tasks onto heterogeneous computing systems. *J Parallel Distrib Comput*. 1999;59(2):107-131.
31. Karatza HD. Scheduling parallel and sequential jobs in a partitionable parallel system. *Int J Simul: Syst Sci Technol*. 2003;4:31-40.
32. Karatza HD. Simulation study of multitasking in distributed server systems with variable workload. *Simul Model Pract Theory*. 2004;12(7–8):591-608.
33. Iosup A, Sonmez O, Anoep S, Epema D. The performance of bags-of-tasks in large-scale distributed systems. In: Proceedings of the 17th International Symposium on High Performance Distributed Computing (HPDC'08), Boston, MA; June 2008:97-108.

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

34. Zikos S, Karatza HD. The impact of service demand variability on resource allocation strategies in a grid system. *ACM Trans Model Comput Simul*. 2010;20(4):19:1-19:29.
35. Stavrinides GL, Karatza HD. Scheduling different types of applications in a saas cloud. In: Proceedings of the 6th International Symposium on Business Modeling and Software Design (BMSD'16), Rhodes, Greece; June 2016:144-151.
36. Zhu X, Qin X, Qiu M. Qos-aware fault-tolerant scheduling for real-time tasks on heterogeneous clusters. *IEEE Trans Comput*. 2011;60(6):800-812.
37. Karatza HD. Performance of gang scheduling strategies in a parallel system. *Simul Model Pract Theory*. 2009;17(2):430-441.
38. Tang HK, Ramanathan P, Morrow K. Inserting placeholder slack to improve run-time scheduling of non-preemptible real-time tasks in heterogeneous systems. In: Proceedings of the 27th International Conference on VLSI Design and 13th International Conference on Embedded Systems 2014, Mumbai, India; January 2014:168-173.
39. Stavrinides GL, Karatza HD. Scheduling real-time jobs in distributed systems - simulation and performance analysis. In: Proceedings of the 1st International Workshop on Sustainable Ultrascale Computing Systems (NESUS'14), Porto, Portugal; August 2014:13-18.
40. Buttazzo GC. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. 3rd ed. New York, NY: Springer; 2011.
41. Kim KH, Beloglazov A, Buyya R. Power-aware provisioning of cloud resources for real-time services. In: Proceedings of the 7th International Workshop on Middleware for Grids, Clouds and E-Science (MGC'09), Urbana Champaign, IL; November 2009:1-6.
42. Van den Bossche R, Vanmechelen K, Broeckhove J. Online cost-efficient scheduling of deadline-constrained workloads on hybrid clouds. *Future Gener Comput Syst*. 2013;29(4):973-985.
43. Liu CL, Layland JW. Scheduling algorithms for multiprogramming in a hard real-time environment. *J ACM*. 1973;20(1):46-61.
44. Banks J, Carson II JS, Nelson BL, Nicol DM. *Discrete-Event System Simulation*. 5th ed. Upper Saddle River, NJ: Pearson; 2010.
45. Law AM. *Simulation Modeling and Analysis*. 4th ed. New York, NY: McGraw-Hill; 2007.
46. Stavrinides GL, Duro FR, Karatza HD, Blas JG, Carretero J. Different aspects of workflow scheduling in large-scale distributed systems. *Simul Model Pract Theory*. 2017;70:120-134.
47. Chen Y, Tsai WT. *Service-Oriented Computing and Web Software Integration: From Principles to Development*. 5th ed. Dubuque, IA: Kendall Hunt Publishing; 2015.

How to cite this article: Stavrinides GL, Karatza HD. Scheduling real-time bag-of-tasks applications with approximate computations in SaaS clouds. *Concurrency Computat Pract Exper*. 2020;**32**:e4208. https://doi.org/10.1002/cpe.4208

15320634, 2020, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.4208 by Oak Ridge National Laboratory, Wiley Online Library on [07/11/2025]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License